

Link Prediction on FB15k-237: Comparing Random, Hard, and Mixed Negative Sampling

Thalia Delhaise, Lenny Briclet, André Fonseca, and Rafael Gufler

RC2526, Knowledge Graphs

Abstract. We study link prediction on the FB15k-237 benchmark using two knowledge graph embedding models: TransE and RotatE. Our current baselines use default Bernoulli negative sampling. However, this strategy mostly generates random negative triples, and many of them are too easy for the model. For the final project, we propose to implement a two-stage negative sampling procedure. First, we generate several negative candidates and score them with the current model. Second, we train with different selection strategies: random negatives, hard negatives, and a mix of both. The goal is to test whether hard negatives really improve link prediction, or whether a combination of random and hard negatives works better.

1 Short Explanation of the Final Project Idea

This section gives a simple overview of what we want to do.

In link prediction, the model receives a partial triple such as $(h, r, ?)$ or $(?, r, t)$ and must predict the missing entity. To train the model, we need positive triples and negative triples. Positive triples are true facts from the knowledge graph. Negative triples are fake facts created by corrupting a true triple.

For example, from the true triple

$(\text{Obama}, \text{born_in}, \text{Honolulu}),$

we can create a negative triple by replacing the tail entity:

$(\text{Obama}, \text{born_in}, \text{Microsoft}).$

This negative is very easy because Microsoft is not a plausible place of birth. The model can learn from it at the beginning, but after some training, such easy negatives do not provide much useful signal.

Our project is about testing better ways of choosing negative triples. The main question is:

Is it better to train a link prediction model with random negatives, hard negatives, or a mix of both?

A hard negative is a false triple that the model currently finds plausible. For example:

(Obama, born_in, New York)

is still false, but it is more plausible than Microsoft. This makes it harder and more useful for training.

However, hard negatives are not always better. If we train only with hard negatives, the model may become unstable or focus too much on confusing examples. This is why we want to compare three strategies:

1. random negatives only;
2. hard negatives only;
3. a mix of random and hard negatives.

This comparison will become the main contribution of our final project.

2 Current State of the Project

We currently have working baselines for TransE and RotatE on FB15k-237.

Both models were trained under the same conditions: embedding dimension $d = 128$, Adam optimizer with learning rate 10^{-3} , batch size 1024, up to 50 epochs, early stopping on validation MRR, CUDA, and seed 42.

The current baselines use default Bernoulli negative sampling. This means that for each positive triple, the sampler decides whether to corrupt the head or the tail, then replaces that entity with another entity.

Table 1 shows our current test-set results.

Table 1. Current test-set results with default Bernoulli negative sampling.

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.152	0.073	0.174	0.304
RotatE	0.261	0.179	0.286	0.424

RotatE performs better than TransE on all metrics. For this reason, our main experiments will focus on RotatE. If time allows, we will also repeat the comparison on TransE.

3 Problem with the Current Negative Sampling

The current Bernoulli negative sampling baseline is useful, but it has one main limitation: it mostly generates random negatives.

Many random negatives are too easy. For example, if the relation is `born_in`, a random corruption may replace a city with a company, a movie, or another unrelated entity. The model can quickly identify these triples as false.

This raises an important question:

Does the model learn better if we train it with harder negative examples?

But we should not assume that hard negatives are always better. A hard negative can also be noisy or too difficult. Therefore, we want to compare different strategies instead of only testing hard negatives.

4 Proposed Contribution

Our proposed contribution is a two-stage negative sampling procedure.

4.1 Stage 1: Generate and Score Negative Candidates

For each positive triple (h, r, t) , we first generate several candidate negative triples.

For tail corruption, we generate:

$$(h, r, t'_1), (h, r, t'_2), \dots, (h, r, t'_n).$$

For head corruption, we generate:

$$(h'_1, r, t), (h'_2, r, t), \dots, (h'_n, r, t).$$

Then we use the current model to score each candidate negative triple. Let $s_\theta(h, r, t)$ be the model score, where a higher score means that the triple is more plausible.

A negative triple with a high score is considered hard because the model currently finds it plausible even though it is false.

4.2 Stage 2: Select Negatives for Training

After scoring the candidate negatives, we choose which negatives to use for training. We will compare three strategies.

Strategy 1: Random-only negatives. We choose negative triples randomly. This is close to the current Bernoulli baseline.

Strategy 2: Hard-only negatives. We choose the negative triples with the highest model scores. These are the negatives that the model currently finds most confusing.

Strategy 3: Mixed random and hard negatives. We choose some random negatives and some hard negatives. For example, if we need k negatives per positive triple, we can choose $k/2$ random negatives and $k/2$ hard negatives.

The mixed strategy is important because hard negatives are not always the best on their own. A mix may give a better balance between stable training and informative examples.

5 Example

Consider the true triple:

(Obama, born_in, Honolulu).

The sampler may generate several negative candidates:

(Obama, born_in, Microsoft),

(Obama, born_in, Paris),

(Obama, born_in, New York),

(Obama, born_in, Apple).

The model scores each candidate. If it gives high scores to Paris and New York, these candidates are hard negatives because the model finds them more plausible.

Then the training strategy decides what to do:

- random-only may choose any candidates;
- hard-only will choose the highest-scored candidates;
- mixed will choose some random candidates and some high-scored candidates.

This allows us to test which type of negative training works best.

6 Experimental Design

Our main experimental comparison will be:

RotatE + random negatives vs. **RotatE + hard negatives** vs.
RotatE + mixed negatives.

Everything else will remain fixed: same dataset, same model architecture, same embedding dimension, same optimizer, same batch size, same number of epochs, and same filtered evaluation protocol.

The goal is to isolate the effect of the negative sampling strategy.

Table 2. Planned ablation for negative sampling strategies.

Experiment	Negative strategy	Purpose
A1	Random-only	Current baseline / control
A2	Hard-only	Test if hard negatives improve training
A3	Mixed random + hard	Test if a balanced strategy works best

6.1 Main Ablation

The main ablation is shown in Table 2.

The random-only setting is our control experiment. The hard-only setting tests whether the model learns better from difficult examples. The mixed setting tests whether combining both types of negatives gives the best performance.

7 Grouped Evaluation

In addition to global MRR and Hits@10, we will evaluate the effect of the sampling strategy on different parts of the graph.

We will report results for:

- rare relations vs. frequent relations;
- low-degree entities vs. high-degree entities;
- head prediction vs. tail prediction.

All relation frequencies and entity degrees will be computed only from the training set. This avoids using information from the validation or test sets when defining the groups.

This grouped evaluation will help us answer more precise questions:

- Do hard negatives help more on rare relations?
- Do mixed negatives help low-degree entities?
- Is the effect different for head prediction and tail prediction?

8 Development Plan

The remaining development work is divided into the following steps.

8.1 Step 1: Clean the Baseline Code

We already have the baseline training pipeline for TransE and RotatE. The first step is to make sure that the baseline code is clean and easy to reuse.

We need to keep:

- the same dataset split;
- the same evaluation code;

- the same metric logging;
- the same training configuration.

This is important because the new experiments must be comparable to the current baseline.

8.2 Step 2: Implement Candidate Generation

For each positive triple, we need to generate several negative candidates. We will start with $n = 32$ or $n = 64$ candidates per positive triple.

The candidate generation should follow the same head/tail corruption logic as the baseline, so the comparison remains fair.

8.3 Step 3: Score Candidate Negatives

After generating the candidates, we will pass them through the current model and compute their scores.

The score tells us how plausible each negative triple is according to the model.

High score means hard negative.

8.4 Step 4: Implement the Three Selection Strategies

We will implement three selection strategies:

1. random-only selection;
2. hard-only selection;
3. mixed random/hard selection.

For the mixed strategy, we will start with a simple 50/50 ratio: half random negatives and half hard negatives.

If time allows, we can also test other ratios such as 70/30 or 30/70.

8.5 Step 5: Train RotatE with Each Strategy

We will train RotatE using each negative sampling strategy.

The main runs will be:

- RotatE + random-only negatives;
- RotatE + hard-only negatives;
- RotatE + mixed random/hard negatives.

We will keep all other hyperparameters fixed.

8.6 Step 6: Evaluate and Compare Results

We will evaluate all models using filtered MRR and Hits@10.

We will compare:

- global performance;
- performance by relation frequency;
- performance by entity degree;
- head prediction and tail prediction separately.

8.7 Step 7: Final Discussion

At the end, we will discuss whether hard negatives are actually useful, and whether the mixed strategy gives a better trade-off than random-only or hard-only training.

9 Expected Outcome

At the end of the project, we want to answer the following question:

For link prediction on FB15k-237, is it better to train RotatE with random negatives, hard negatives, or a mix of both?

We expect the mixed strategy to be a strong candidate because it combines:

- random negatives, which keep training stable;
- hard negatives, which provide more informative training examples.

However, we will not assume that the mixed strategy is best. The goal of the project is to test this experimentally.

10 Conclusion

Our current D2 baselines use Bernoulli negative sampling, which is a useful starting point but mostly produces random negatives. For the final project, we propose to implement and evaluate a two-stage negative sampling procedure.

The sampler first generates and scores several negative candidates. Then it selects negatives using one of three strategies: random-only, hard-only, or mixed random/hard. This gives us a clear ablation and allows us to test whether hard negatives really improve link prediction, or whether a balanced mix is better.

This contribution is simple enough to implement within the project scope, but it also gives a meaningful experimental question and a clear final comparison.